Университет ИТМО

Инструментальные средства разработки программного обеспечения

Создание расширения для VS Code

Отчёт по лабораторной работе 3

Code Recall

Выполнил: Лукьянов Егор Юрьевич
Группа: M3113
Дата: 30 октября 2026 года

Цель и результат работы

Цель работы — изучить механизм расширения среды разработки и реализовать собственный плагин. Разработан Code Recall — тренажёр восстановления выделенного кода по памяти. Пользователь получает пустую вкладку для попытки и сравнивает результат с сохранённым оригиналом.

Назначение расширения

Code Recall помогает готовиться к защите лабораторных работ: можно выделить функцию, написать её самостоятельно и увидеть текстовые различия. Исходный файл остаётся неизменным. Первая версия поддерживает одну активную тренировку и не отправляет код внешним сервисам.

Материалы проекта

Общедоступный репозиторий исходного кода:
https://github.com/chydachyo/code-recall
Установочный пакет: code-recall-0.0.1.vsix.
Основной исходный файл: src/extension.ts.
Авторство и возможности приведены в README.md.

Архитектура VS Code и расширений

Редактор предоставляет интерфейс пользователя и Extension API для работы с документами, командами, вкладками и уведомлениями. Код расширений выполняется в среде Extension Host. Расширение объявляет свои возможности в манифесте package.json и использует API, не изменяя исходный код редактора.

В contributes.commands объявлены четыре команды Code Recall. VS Code активирует расширение при обращении к одной из них. Функция activate создаёт RecallTrainer и связывает идентификаторы команд с обработчиками. Зарегистрированные ресурсы помещаются в context.subscriptions для освобождения при отключении расширения.

Команда	**Действие**
code-recall.start	Проверяет выделение, сохраняет снимок оригинала и открывает пустую попытку.
code-recall.compare	Открывает встроенное сравнение оригинала и попытки.
code-recall.resume	Возвращает пользователя к редактируемому документу попытки.
code-recall.finish	После подтверждения закрывает вкладки тренировки; предлагает сохранение несохранённого текста.

Хранение состояния и сравнение

Training хранит текст оригинала, идентификатор языка, URI оригинала и документ попытки. Оригинал предоставляется через TextDocumentContentProvider со схемой code-recall и доступен только для чтения. Попытка создаётся через workspace.openTextDocument и редактируется как обычный текстовый документ.

Команда vscode.diff показывает оба документа. Проверка полного совпадения сравнивает текст после нормализации CRLF в LF. Это не проверка семантики: другое корректное решение может отличаться от эталона. Обработчики не запускают компилятор пользовательского кода.

Разработка и проверка

Заготовка создана генератором Yeoman generator-code: TypeScript, сборщик esbuild, менеджер npm. Исходная команда Hello World заменена командами тренажёра. В манифест добавлены контекстное меню редактора, описание, авторство и ссылка на репозиторий.

Выполнены npm.cmd run compile и npm.cmd run test:logic. Проверка типов TypeScript, ESLint и сборка завершились успешно. Автоматическая проверка с имитацией API охватывает пустое и множественное выделение, неизменность снимка, отмену замены, сравнение, возврат, отмену закрытия и завершение.

Вручную в VS Code проверены создание попытки, подсветка отличий a + b и a * b, а также уведомление о полном совпадении после исправления. Интеграционный тестовый запуск отдельного VS Code был заблокирован ограничениями среды до выполнения тестов; он не учитывается как успешно пройденный.

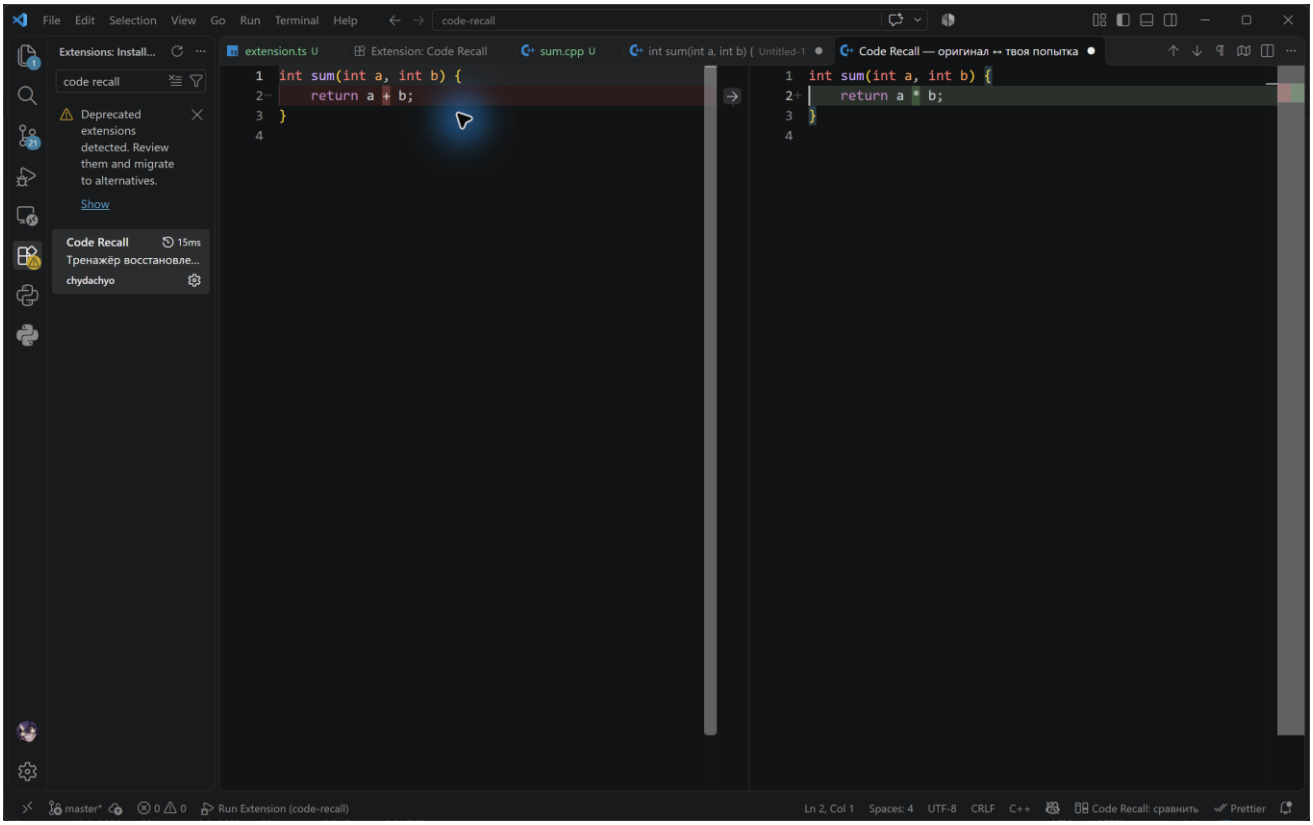


Рисунок 1 — сравнение оригинала и попытки в установленном Code Recall

Ограничения и вывод

Тренировка хранится в памяти текущего окна до перезагрузки или отключения расширения. Автодополнение других расширений не отключается. Получен работающий плагин с четырьмя командами, отдельной попыткой и сравнением; установка не требует запуска отладки F5.

Документация: https://code.visualstudio.com/api/references/vscode-api
Упаковка: https://code.visualstudio.com/api/working-with-extensions/publishing-extension